

Lecture 8

How LLM works

①

- A Large Language Model is an AI model designed to understand & generate human-like text.
- It has been trained on huge volumes of text data like books, websites & articles.

LLMs ^{or machines} don't understand plain text like "Hi, How are you?"

→ High-Level working of an LLM :-

Step

①

Tokenization

- first, the sentence is broken into tokens (like words or sub-words)

then each token is converted into a number using a vocabulary dictionary

A token is the smallest unit an LLM understands. It can be a word, part of a word or even punctuation

eg:- "Hello" → ["Hel", "lo"] (two tokens in some tokenizers)

But in some tokenizers it is only one token "Hello".

→ Different LLMs use different tokenization strategies such as:-

- (i) word-based
- (ii) subword-based (like Byte-Pair Encoding or BPE)
- (iii) character-based

OpenAI's models use a Byte-Pair Encoding tokenizer called tiktoken

text $\rightarrow$ tokenizer $\rightarrow$ tokens $\rightarrow$ token IDs

"~~hello~~ Jenny" $\rightarrow$ ~~["hello", "Jenny"]~~
 $\rightarrow$ ["~~hello~~", "J", "enny"] $\rightarrow$ [15339, 503, 18314]
 $\downarrow$
 token IDs

Step 2 Embedding

$\rightarrow$ Converting tokens to vectors of numbers that the neural network can understand

NOTE! - token ID and embeddings are not same.
 token ID is used to fetch its embedding

- Embeddings are high-dimensional vectors (e.g. 768-d for GPT-2)
 eg:- $[0.23, -0.15, 0.98, \dots, 0.01]$

- Every token ID is looked up in an embedding table like this:-

Embedding[token_id] $\rightarrow$ $[0.23, -0.15, 0.98, \dots, 0.01]$

So Embedding[15339] $\rightarrow$
 Embedding[503] $\rightarrow$
 Embedding[18314] $\rightarrow$ } each one will have its separate embedding vector

As GPT use Transformer architecture. So the embedding layer maps token IDs $\rightarrow$ embedding vectors.

Question:- Are these embeddings vectors (values) stored somewhere?

So these embeddings come from a pre-trained embedding table that is learned during training of the model.

[Think of it like a giant excel sheet]

Imp In LLMs like GPT, there are two main phases:

① Training: - Teaching the model the structure, facts, reasoning & patterns in human language by feeding it lots of text. It learns language patterns

② Inference: - Use the already trained model to generate responses, answer questions, complete texts etc.

Each GPT model (like GPT-2, GPT-3, GPT-4) has its own vocabulary (token dictionary)

GPT-2, GPT-3 has around 50,257 tokens

GPT-4 ~ 100,000 tokens [they haven't disclosed it]

& Embedding vectors for these tokens. During training, the model learns a vector representation (numerical form) for each token. These are stored in an embedding matrix.

Size = vocab_size * embedding_dim

Size = 50k * 768 (for GPT-2 small)

How embeddings are generated?

↳ These are not written manually or pre-defined; they are learned during training using backpropagation.

How training is done?

→ will cover in next lecture.

IMP
NOTE:- Embeddings are special vectors (list of numbers) that capture ~~the~~ meaning.

→ If "man" is $[0.013, -0.12, \dots, 0.75]$ & "woman" is $[0.012, -0.13, \dots, 0.70]$ means they are close in meaning

So these embedding vectors values pertaining to semantic & syntactic information.

⇒ Semantic info:- means meaning of words & their relationship with other words. e.g - Doctor, nurse, hospital, patient these words are related semantically. In embedding space, they might be represented by vectors that are close to each other

⇒ Syntactic info:- arrangement of words and phrases to create well-formed sentences in a language

eg:- in English typical word order is Subject-verb-Object

Jenny loves teaching

So syntactic info helps in identifying correct order of words to make a meaningful sentences

These things are learned during training so the model knows "doctor" & "nurse" are related.

Step 3:- Transformer Architecture

⑤

↳ Brain behind LLM is transformer

So after getting embeddings of each token, we feed these embeddings to model (transformer) but before that we need to add positional information.

⇒ As Transformers processes all tokens at once & look at all parts of the input to understand context, so before sending these embedding to Multi-Head Self Attention, we add position info manually. So the positional encoding helps the model understand order of tokens

So

Embedding vector + Position Vector = Final Input to Transformer

There are 2 methods to add positions:-

① Sinusoidal Positional Encoding (fixed, math-based)
↳ used in original transformer paper

② Learned Positional Embeddings:

↳ used in models like BERT, GPT

→ This combined vector is then passed through the transformer block which starts with the most imp. part:- Multi-Head self Attention. It means each word looks at every other word, including itself to understand the context.

encoding:-

Now these tokens are sent to Encoder Layers.

LLMs have many layers of transformers stacked. & Each layer performs a special operation on the incoming tokens.

In every layer, what happens is:-

- (1) Self-Attention - Each word looks at every other word to understand context
- (2) Add & Norm - helps combine info & stabilize understanding
- (3) Feed forward Network - Each word refines its meaning using a small neural network
- (4) Add & Norm - to ensure stability & improve learning, adding the input again back to output of feedforward Network & then we normalize the result.

So imagine each layer a kind of workshop where tokens get refined, reshaped based on surrounding context of info.

→ GPT3 has 96 Transformer layers [175 Billion parameters]

⇒ [That's like 96 rounds of intense classroom discussion before predicting the next word]

IMP

NOTE:- A full transformer (like encoder-decoder models such as T5 or the original transformer used in translation) have both encoder & decoder phase but GPT uses only the decoder.

⑦

The decoder's job is to generate output tokens, one token at a time

using:

- (i) what the encoder has understood (encoder's final output)
- (ii) what it has already generated so far (previous output token)

→ Que. is: - But at starting we have only input not output token?

Ans: - So at first only special token <START> is given to indicate the start of sentence as output embeddings.

Decoder has 3 main layers:-

① Masked Multi-Head self Attention:-

↳ means it look at the words that has been generated so far, but not the future ones

② Cross Attention (Encoder-decoder Attention)

↳ means it attends to the encoder's output, understands the meaning of the original sentence (input)

This layer helps the decoder to know "what did the encoder understand?"

③ Feedforward & final Prediction:-

↳ Each token (word) refines its meaning going through a small neural network.

At the very end we get the final hidden vector for each token position. [output token embeddings]

So decoder-only transformers produces a corresponding output vectors for each token embedding

⑧

Step 4 :- Text Generation (Prediction)

- As the output of the last layer of decoder-transformers is a prediction vector per position (but we care about the last position, since we want the next word).

Now final Linear + Softmax

Linear layer takes the final hidden vector & converts it into a large vector of size = vocabulary size (e.g. 50257)

Output :- scores for every word in the vocabulary (called logits)

Now Softmax layer converts these logits into probabilities, & then GPT can decide which word is most likely to come next.

eg - If GPT is trying to continue the sentence: -

The sky is - - -

Some possible next tokens with probabilities

<u>Token</u>	<u>Probabilities</u>
blue	0.70 $\Rightarrow$ <u>highest probability</u>
Jenny	0.001
pink	0.25
dusty	0.66

The model continues generating tokens until a stopping criterion is met such as end-of-sentence <eos> or a maximum token limit.